NAME

sumtag - stamp files with an XXH3 checksum and metadata for corruption and duplicate detection

SYNOPSIS

sumtag [*OPTIONS*] *DIRECTORY* [*DIRECTORY*...]

DESCRIPTION

sumtag recursively scans each *DIRECTORY*, computes an XXH3 checksum of every file's contents, and stores the checksum together with metadata in an extended attribute named **user.sumtag** on the file. The recorded metadata makes it possible to later detect silent data corruption and to find duplicate files. Those higher-level analyses are left to other tools, with one exception built in here: the single-pass integrity check offered by **--verify**.

An explicit action option is required: every run must name what it does with one of **--sum**, **--verify**, **--remove**, **--import**, **--locate**, **--prune-dirs**, or **--prune-all**. A bare **sumtag** *DIRECTORY* is an error. The plain stamping action is **--sum**, which works with or without a database (see OPTIONS).

At least one *DIRECTORY* is also required; there is no current-directory default. Any operation that scans a directory requires an explicit directory argument, so that a recursive operation is never aimed at the working directory by omission. Pass **.** explicitly to scan the current directory.

For each file, sumtag reads any existing **user.sumtag** attribute and compares the recorded modification time to the file's current mtime. If they match, the file is considered up to date and is skipped. Otherwise the file's contents are hashed and the attribute is written.

Within each directory, files are processed in ascending alphabetical order and subdirectories are then recursed into in the same order, so the processing order can be followed against an **ls**(1) listing. This holds in every mode.

Symbolic links are not content: a symlink encountered during traversal is never followed, hashed, stamped, verified, or removed, in any mode. A *DIRECTORY* named on the command line may itself be a symbolic link, which is honoured; nothing found beneath it is followed.

RE-HASHING

A file is hashed, or re-hashed, when any of the following hold:

- the **--force** option was given;
- the **user.sumtag** attribute is absent or unreadable;

- the stored modification time is older than the file's current mtime;
- the stored schema version has a lower major number than the running software.

Freshness is algorithm-agnostic: a file with any current digest, regardless of which algorithm produced it, counts as up to date. A stored digest is never recomputed merely because a different algorithm is now preferred; when a re-hash does happen for one of the reasons above, it replaces the file's single stored digest rather than keeping the old one alongside it.

OPTIONS

-n, **--dry-run**
Scan and report what would be done, but write nothing -- no extended attributes and no database rows. Output matches a real run. Cannot be combined with **--force**.

-q, **--quiet**
Suppress normal output. Repeat (**-qq**) to also suppress error messages on standard error. Mutually exclusive with **--verbose**.

-v, **--verbose**
Explain the decision made for each file, including skips. Without this option, each file touched is announced by its bare path alone. Repeat (**-vv**) for deep internal detail intended for debugging. Mutually exclusive with **--quiet**.

--progress
Show a live within-file progress indicator for large files. If it appears together with **-q**, whichever option appears last on the command line wins, and a warning is printed to standard error. The indicator's line shows the file's size, the current hashing rate, a bar, the percentage complete, elapsed time, and an ETA, e.g.:
99.3GiB 40.1MiB/s [==========>] 84% 0:05:12 ETA 58s

The bar is the only field whose width varies: it absorbs whatever room the fixed-width fields leave, and resizes live when the terminal window is resized.

--si Display **--progress** sizes and rates using decimal (SI, powers-of-1000: **kB**/**MB**/**GB**) units instead of the default binary (powers-of-1024: **KiB**/**MiB**/**GiB**) units.

-f, **--force**
Re-hash every file unconditionally, ignoring any existing metadata. Cannot be combined with **--dry-run** or **--verify**. Combined with **--import** or **--locate**, overrides their default refusal to compute (see below). This option does *not* override **@sumtag-ignore** markers; use **--no-ignore** for

that.

--database *VALUE*

Name a database to act on. *VALUE* is a SQLite file path, or a scheme*://* DSN. A value matching the pattern ‘**^[a-z][a-z0-9+.-]*://**’ is treated as a DSN (**mysql://**, **postgresql://**); any other value, including one that merely contains a colon, is a SQLite file path. Only SQLite is implemented; a DSN is recognised and rejected as not yet supported. Database paths are stored relative to the file’s mount point so they survive remounting at a different location. **--database** takes no action by itself; it requires at least one of **--sum**, **--import**, **--locate**, **--prune-dirs**, or **--prune-all** to say what to do with it.

--sum

The stamping action: hash or re-hash each file per the normal mtime-based decision (see **RE-HASHING)** and write the extended attribute. With **--database**, the result is additionally mirrored into the database; without one, only the extended attribute is written.

--import

Do not compute any checksum by default; only copy metadata already present in each file’s extended attribute into the database. Requires **--database**. Files lacking usable metadata are skipped and reported. Useful for populating a database from an archive that was hashed on a previous run. Combined with **--force**, computes and mirrors every file instead of only propagating what already exists. Combined with **--sum**, is redundant (the normal decision already computes and mirrors) but not an error. Cannot be combined with **--verify**.

--locate

Stat every file visited and write filesystem metadata (size, permissions, ownership, link count, device, and timestamps) to the database, regardless of whether xattr work was done. Implies **--import**: existing extended-attribute metadata is propagated as well, so **--locate** alone does everything **--import** alone would, plus stat columns. Requires **--database**. Combined with **--force**, computes checksums for every file too, the same override **--import** accepts. Useful as a periodic filesystem inventory pass (analogous to **updatedb**).

--verify

Read-only integrity check: recompute each file’s checksum and compare it to the stored value, reporting any disagreement. Nothing is written, even on a mismatch. A mismatch while the stored modification time still matches the file’s mtime indicates silent corruption; a mismatch accompanied by a newer mtime indicates an ordinary modification whose stamp is merely stale. Files with no usable metadata are reported as unverifiable. Cannot be combined with **--database**, **--sum**, **--import**, **--locate**, or **--force**; combining with **--dry-run** is a redundant no-op and is permitted. See **EXIT STATUS** below.

--remove

Remove the **user.sumtag** xattr from every file in the tree. A testing/reset utility, not a data-integrity primitive: no mtime comparison, no hashing, just an unconditional delete of whatever attribute happens to be present. Files with no attribute are left alone. Combined with **--dry-run**, previews which files would lose their stamp without removing anything. Cannot be combined with **--database**, **--sum**, **--import**, **--locate**, **--verify**, or **--force**.

--prescan

Walk the tree once up front to count the files that will be checksummed and their total size, then prefix each hash/verify announcement with an *nnn*/*mmm* file counter and a bytes-so-far/total counter, each followed by its whole-number percentage in parens (right-padded to three digits so the columns never jitter), e.g.:

042/137 (31%) 118.2MiB/4.2GiB (3%) hash /backup/vault/photo0042.dng (reason)

The counted set mirrors whichever mode is running: for the normal hash/stamp pass it is the files the re-hash decision (or **--force**) will actually hash; for **--verify** it is every file with a usable stored digest. During the counting walk each directory visited is announced before any file metadata inside it is read (the bare directory path, or *prescan path* with **-v**; suppressed by **-q**), so the otherwise-silent up-front pass shows its own progress on a large tree. The prescan pass is a prediction from a separate walk, not a guarantee -- counts can drift if the tree changes between the two passes. On a **--database** run (and not under **-n**), the counted totals are also stored in the database as its single prescan summary, replacing any previous one; that summary is what **--db-prescan** later consumes. Cannot be combined with **--remove**.

--db-prescan

Like **--prescan**, but load the counters' totals from the summary a previous **--prescan --database** run stored, instead of walking the filesystem -- on a very large tree this replaces an hours-long counting walk with a database read. The stored totals are used only if they were counted for the same scan roots (compared after path normalization) and under the same counting options (**--sum**/**--import**/**--locate**, **--force**, **--exclude**, **--no-ignore**); a missing or non-matching summary is an error before any work begins (exit status 2), never a silent fall-back to a filesystem walk. What was loaded is announced at startup. The progress report is an approximation by consent: the per-file counter tracks what actually happens this run, so it may overshoot the stored total (the tree changed) or the run may finish below it (some of the counted work was already done -- the typical case when resuming an interrupted scan). The stored data is display-only and never influences which files are hashed. Requires **--database**; cannot be combined with **--prescan** or **--remove**.

--prune-dirs

Reconcile the database with a changed filesystem: collect every directory the database knows under the given roots (the distinct parent directories of its file rows), check each one with a single **stat**(2), and delete the rows of directories that no longer exist. A deleted directory takes exactly its

resident file rows with it; nothing is deleted recursively, because a vanished subdirectory is discovered by its own check -- every known directory is in the list. The filesystem is never modified. Each *DIRECTORY* named on the command line must exist -- a missing root is an error (exit status 2) before the database is even opened, so an unmounted drive can never read as a mass deletion -- and only rows recorded under the root's current mount point are candidates. The *DIRECTORY* argument is therefore not what is scanned (the walk is over the database); it is the operation's safety scope. Naming a root that exists is the evidence that the filesystem its rows describe is actually mounted and live, and it confines the reconciliation to that subtree -- so a database that mirrors several drives can be pruned one drive at a time while the others sit unplugged. The database itself must already exist; it is never created by this option. A directory deleted from the filesystem but moved elsewhere is pruned at its old location like any other deletion; files deleted from a directory that still exists are not noticed (that is **--prune-all**'s job). Combined with **--dry-run**, previews what would be pruned without deleting anything. With **--progress**, a live *nnn*/*mmm* counter reports directories checked against the total known at the start. Requires **--database**. Cannot be combined with **--sum**, **--import**, **--locate**, **--verify**, **--remove**, **--force**, **--prescan**, **--db-prescan**, **--exclude**, or **--no-ignore** (the last two govern filesystem traversal, and this option walks the database, not the filesystem). See **EXIT STATUS** below.

--prune-all

Everything **--prune-dirs** does, and additionally check every file row in directories that still exist, with one **lstat**(2) each, deleting the rows of files that no longer exist or are no longer regular files (a path that turned into a directory or symbolic link is not the stamped file). The directory pass runs first, so a vanished directory still costs a single **stat**(2), never one per resident row; the extra cost over **--prune-dirs** is one **lstat**(2) per row in surviving directories only. A moved file is pruned at its old location like any deletion; nothing content-based is checked -- no hashing, ever. Same requirements, conflicts, previews, progress counter (the unit becomes paths: directories plus file rows), and exit status as **--prune-dirs**; giving both together is redundant but not an error.

--no-ignore

Disregard all **@sumtag-ignore** marker files for this run, processing every directory regardless of markers. This option governs markers only; it has no effect on **--exclude**.

--exclude *PATTERN*

Skip any file or directory whose name matches the shell glob *PATTERN*. Matching is against the basename only, case-sensitively; a matching directory is pruned along with its entire subtree, exactly as an **@sumtag-ignore** marker would prune it. May be repeated; a name matching any given pattern is excluded. The exclusion applies in every mode, is not overridden by **--force**, and is unaffected by **--no-ignore**, which governs markers only. A *DIRECTORY* named on the command line whose own name matches a pattern is skipped, with a warning printed to standard error.

--version
Print version information and exit.

-h, **--help**
Print a usage summary and exit.

IGNORE MARKERS

A directory containing a file named **@sumtag-ignore** is exempted along with its entire subtree: sumtag does not descend into it, and nothing beneath it is read, hashed, stamped, or mirrored to a database. The marker file itself is never hashed. Only the file's presence matters; its contents are ignored and reserved for future use. The exemption applies in every mode and takes precedence over **--force**; use **--no-ignore** to override it. A marker found at a directory named directly on the command line is honoured, with a warning printed to standard error.

For per-run exclusion from the command line, without placing a marker file, see **--exclude**.

RUN SUMMARY

Every run ends with a brief summary block on standard output (suppressed by **-q**): the number of files the mode processed and their cumulative size (headlined **hashed**, **would hash**, **imported**, **verified**, **removed**, or **pruned** to match the mode; **pruned** counts directories and file rows, followed by a **checked** line with the total directories examined -- under **--prune-all**, files too), any nonzero **skipped**, **errors**, **CORRUPT**, **stale**, or **unverifiable** counts, the database in use (when **--database** was given), and the scanned directories. Byte figures honour **--si**.

Interrupting a run with Ctrl-C prints the same summary, prefixed by an **interrupted** line, instead of a Python traceback; the counters cover only files whose work had already completed. The process then exits **130** (128 + SIGINT, the shell convention).

EXIT STATUS

Without **--verify**, **--prune-dirs**, or **--prune-all**, sumtag exits **0** on success and non-zero if an error prevented completion.

With **--verify**:

0 all checked files verified intact.

1 one or more checksum mismatches (corruption) were found.

2 unreadable files or other errors prevented a complete check.

With **--prune-dirs** or **--prune-all**, the same scheme, applied to staleness: **0** means nothing was stale (the database already matches the filesystem), **1** means stale directories or file rows were found and pruned (or would be pruned, under **--dry-run**), and **2** means errors prevented a complete check (including a missing scan root or database).

In any mode, a run interrupted by Ctrl-C exits **130** (see RUN SUMMARY).

FILES

user.sumtag
Extended attribute holding each file's checksum and metadata as UTF-8 JSON.

@sumtag-ignore
Marker file that exempts its directory and subtree (see IGNORE MARKERS).

EXAMPLES

Stamp the current directory:

```
sumtag --sum .
```

Stamp two archives:

```
sumtag --sum /data /backup
```

Preview without writing anything:

```
sumtag --sum -n /data
```

Stamp and also mirror into a SQLite database:

```
sumtag --database /var/lib/sumtag.sqlite --sum /data
```

Import existing metadata into a database without re-reading files:

```
sumtag --database /var/lib/sumtag.sqlite --import /data
```

Take a filesystem inventory pass, capturing stat metadata alongside any existing checksums:

```
sumtag --database /var/lib/sumtag.sqlite --locate /data
```

Verify an archive against its stored checksums:

sumtag --verify /backup

Resume an interrupted database run without repeating its hour-long counting walk, reusing the totals the interrupted run’s --prescan stored:

sumtag --sum --database /var/lib/sumtag.sqlite --db-prescan /data

Stamp an archive, skipping DVD rip payloads:

sumtag --sum --exclude ’*.vob’ --exclude VIDEO_TS /data

After deleting directories from an archive, drop their stale database rows (previewing first):

sumtag --prune-dirs -n --database /var/lib/sumtag.sqlite /data
sumtag --prune-dirs --database /var/lib/sumtag.sqlite /data

NOTES

Timestamps are recorded in UTC with microsecond precision. Mtime comparisons truncate both the stored and live values to the same precision, so the comparison is symmetric.

SEE ALSO

grouper(1), **dedupe**(1), **xattr**(1), **getfattr**(1), **setfattr**(1), **cksum**(1)

AUTHOR

The sumtag authors.